

TECHNICAL DESIGN: MULTI-SOURCE CANDIDATE DATA TRANSFORMER

Author: Vivek Sharma | Email: vivek.sharma@gmail.com | Deliverable: Technical Design One-Pager

1. ARCHITECTURAL PIPELINE OVERVIEW

A modular, deterministic pipe-and-filter architecture digests candidate profiles:

- **Detect & Extract:** Ingests resumes (PDF/TXT), Recruiter CSVs, and ATS JSONs. If a GitHub profile URL is detected, the pipeline automatically spins up a background thread to call the GitHub API and ingest the candidate's public details.
- **Normalize:** Cleans data: validates and formats phone numbers to E.164, parses locations to city/region/country (mapping country to ISO-3166-1 alpha-2), standardizes dates to YYYY-MM, and matches skills against a canonical database using semantic sentence embeddings (all-MiniLM-L6-v2).
- **Merge:** Resolves overlapping entities via priority ranking and deduplication heuristics.
- **Scorer:** Assigns field and record-level confidence scores.
- **Project & Validate:** Dynamically maps fields using JSONPath syntax, and runs a secondary runtime validation on the output.

2. CANONICAL OUTPUT SCHEMA & NORMALIZATIONS

The system standardizes data into a strict **Canonical Candidate Model**:

- **candidate_id:** Deterministic SHA-256 hash of sorted emails + lowercase name.
- **full_name:** Cleaned and converted to title case.
- **emails / phones:** Deduped lists; phones are validated and stored in E.164 format.
- **location:** Structured object: {city, region, country} (ISO country codes).
- **links:** Structured object: {linkedin, github, portfolio, other[]}
- **skills:** Normalized canonical skill names with field confidence & source history.
- **experience / education:** Nested arrays containing standard date and degree details.
- **provenance:** Field-level audit trail tracking the source and method used for each value.
- **overall_confidence:** Score representing merged data reliability.

3. MERGE & CONFLICT-RESOLUTION POLICY

- **Conflict Resolution:** Single-value fields are resolved using priority weighting (Resume: 5, LinkedIn: 4, GitHub: 3, ATS: 2, CSV: 1). If there is a tie, the more complete/longer string is chosen.
- **Fuzzy Deduplication:** Lists are merged. Education items standardize degree abbreviations (e.g. 'B.E' -> 'Bachelor of Engineering') and perform acronym mapping (e.g. 'SPPU' -> 'Savitribai Phule Pune University').
- **Classification Correction:** If a candidate's field-of-study (e.g. 'Computer Engineering') is misclassified as the institution, the merger swaps it into the 'field' key and keeps the real university name.
- **Scorer:** Calculates field confidence based on source reputation, value presence, and extraction method (LLM vs regex). Overall confidence averages only active, non-zero scored fields.

4. RUNTIME CUSTOM-OUTPUT PROJECTION

Users can project the canonical profile to any custom structure using a JSON config:

- **Path Resolution:** Supports recursive dot/index queries (e.g. `skills[0].name` or `emails[0]`).
- **Type Coercion:** If a path resolves to an object (e.g. a skill dictionary) but the target type is 'string', it automatically extracts the primary textual field (e.g., 'name') to avoid mismatches.
- **Dynamic Schema Validation:** Reconstructs a JSON Schema at runtime from the projection settings (supporting array, object, and type variants like `string[]`) to validate the projected output structure.

5. EDGE CASES HANDLED & TIME TRADE-OFFS

- **Acronym / Substring Matches:** Handled school and degree name variations to merge education records cleanly.
- **LLM Failure Fallback:** Integrated a robust regex fallback parser that extracts all profile sections (name, links, skills, experience, education) from PDF content if API keys are absent.
- **Bracketed Type Notation:** Mapped bracketed types (e.g., `string[]`) to standard JSON Schema arrays.
- **Deliberate Trade-offs (Time Pressure):** Excluded support for temporal overlap checking in experience timelines and manual conflict overrides via the UI (defaults to prioritized merging).